🌷

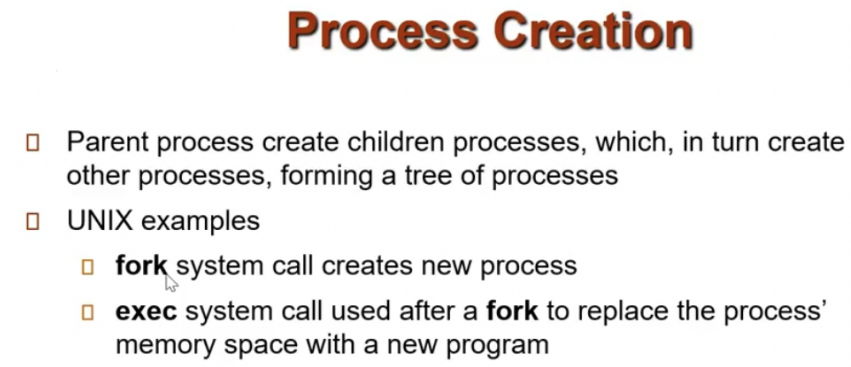
Week 5

✅Process Creation(Fork)

- fork()
  - how fork works?
- Process IDs (pid)
- Things to Note
- Some question
- More Example

Content

✅Process Creation(Fork)

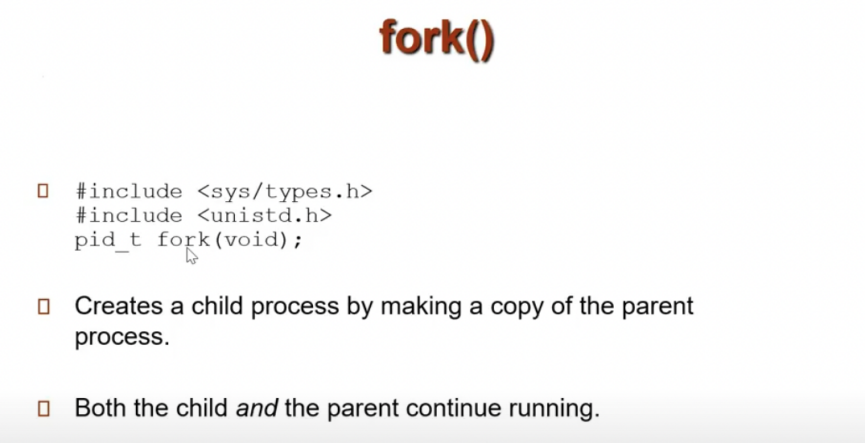


- When you start a process from the program, that process is called “Parent”.
- when you start a new process → that process is called “Parent”, and if you want to do some concurrency program → that parent can start child process.
  - Parent and Child can work in different area of the same program.

        - There are 2 processes that work at at same time in different part of program
    - Benefit, for example, you write a program to find prime number between 1 and million → if you have multi-core cpu and create 2 process, Parent process work from 1 - 50,000, the child process work from 50,001 to 1 million, so this 2 process work in different core → and you will get result faster.

> The way to create the child process is by calling system call “fork”

📍 fork()



- fork didn’t require any parameter, and it return pid_t (in this point, it’s integer number)

Code

Example of fork()

```
/* forkExample.c */
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
int main()
{
 pid_t pid;
 int i = 0;
 pid = fork();
 if (pid > 0) {  /* parent */
  i = 1;
  printf("I am parent i = %d\n", i);
 }
 else {   /* child */
  i = 2;
  printf("I am child i = %d\n", i);
 }
 return 0;
```

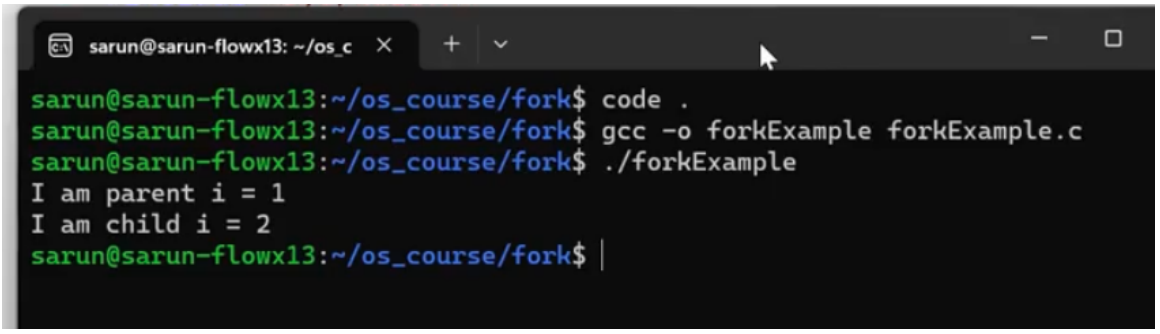
- If you don't know what fork() is, it'll print out 1 line depend on value of pid
  - either I am parent i, I am child i

If it a normal program, it may print out 1 line of output.

The way to compile and run C program

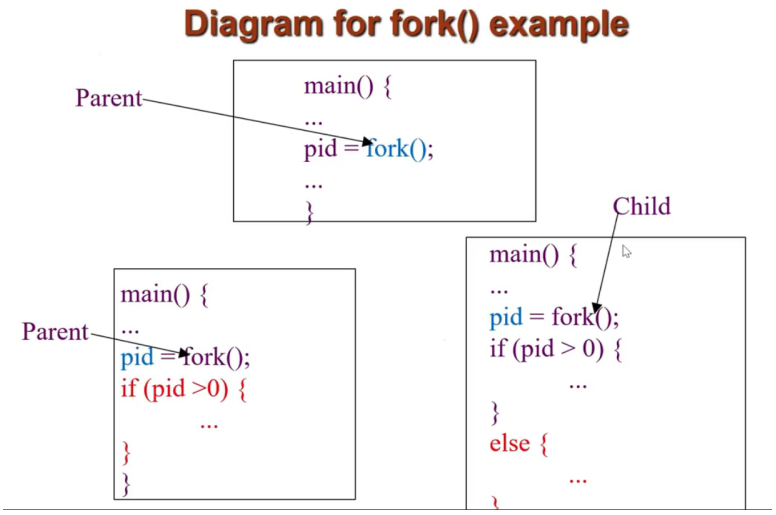
```
gcc -o forkExample forkExampke.c

./forkExample
```

> It'll print out 2 lines, I am parent i = 1, I am child i = 2



because fork() is system call that create a child process.

How fork() work?



When you write this program, you have only 1 process → parent process.

- but when parent process call fork(), now OS create a child process of this process

You now have 2 processes → Parent, child process.

- The idea is fork copy everything from parent to child, it mean it copy the code of parent to child. Now parent and child have the same program → It copy the value of variable of parent to child as well. Also, program counter.

Back

Example of fork()

```
/* forkExample.c */
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
int main()
{
 pid_t pid;
 int i = 0;
 pid = fork();
 if (pid > 0) {  /* parent */
  i = 1;
  printf("I am parent i = %d\n“, i);
 }
 else {   /* child */
  i = 2;
  printf("I am child i = %d\n“, i);
 }
 return 0;
```

- The current value of pid and i will be copy to child as well.
- After the fork, parent and child will start at the same point (program counter).
  - This mean return some value back to variable pid. (it both start here)
- Now, we have 2 copy of pid. → changing value of pid in parent, it won't be effect on child.

👉 For parent, fork() will return a positive number (process id of child).

- Parent will know the process id of the child, it just have created from fork.

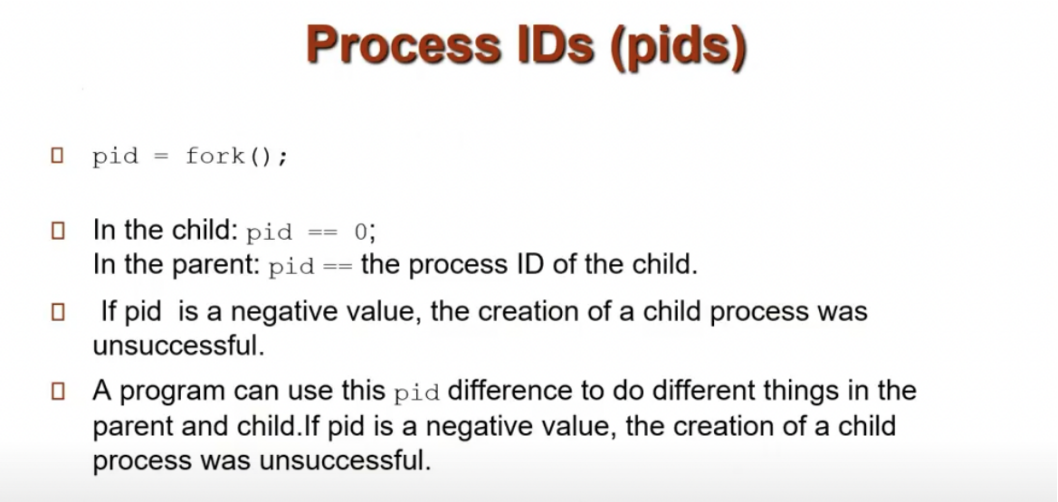
For child, fork() will return 0.

- Child always get 0.

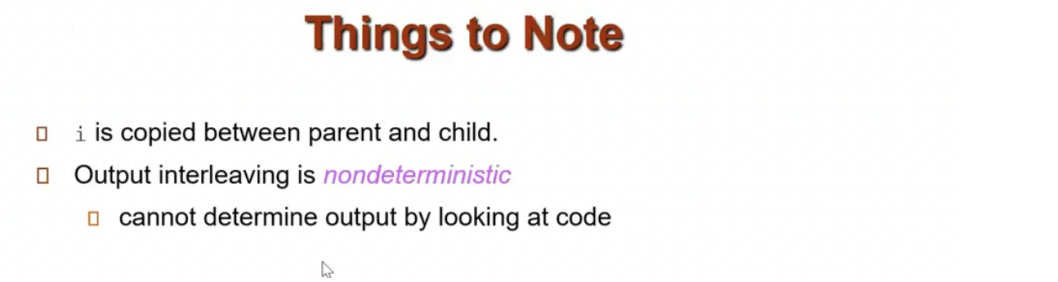
This process can work concurrently. The parent execute in if part, The child execute in else part.

- Each process in a system must have unique process id, so parent will know the process id of the child that it just just created, child always get 0.

📍 Process IDs (pids)



📍 Things to Note



Nondeterministic → it may be different

- The point is when you run program the order of output in unpredictable.
- We don't know which process will get run first → depends on OS
- You can only said the amount of line of output and what it is, not an order.

📍 Some question

- **Can we change pid_t to int?**

```c
#include <sys/wait.h>
int main()
{
  int pid;
  int i;
  pid = fork();
  if (pid > 0) {  /* parent */
    i = 1;
    //sleep(10);
    printf("I am parent i = %d\n", i);
  }
  else {   /* child */
    i = 2;

    printf("I am child i = %d\n", i);
  }
  return 0;
}
```

Same result

```
sarun@sarun-flowx13:~/os_course/fork$ ./forkExample
I am parent i = 1
I am child i = 2
sarun@sarun-flowx13:~/os_course/fork$
```

Yes, you can use int, but your program may not be consider as portable program.

- **Why fork() use pid_t instead of int?**
  - (Java) when you declare integer var, you always ensure that integer will have 4 bytes → java run under it own JVM.
  - (For C for some OS platform) → int may have 2 bytes, 4 bytes
  - If use int, you don't know is it gonna be 2 or 4 bytes, safer way use pid_t, because it will declare inside header file.
  - (not sure which one)

```c
/* forkExample.c */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
```

👉 Ajarn expect us to check error (if it can't be fork()) → when the system resources full → OS won't allow parent to fork a child anymore.

👉 The parent can fork any number of the child process as long as the system resource is not full.

👉 Child process can fork it own child.

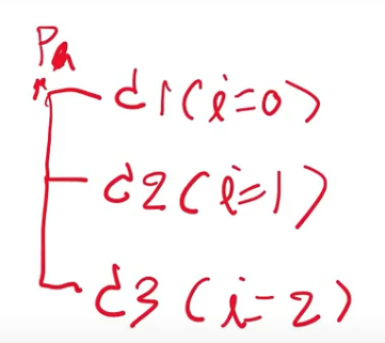
📍 More Example

- 🌱**Example 1**

```c
int main()
{
  pid_t pid;
  int i;
  for (int i = 0; i < 3; i++) {
    pid = fork();
    if (pid == 0) {
      printf("child i = %d\n", i);
      exit(0);
    }
    else {
      printf("parent i = %d\n", i);
    }
  }
  return 0;
}
```

```
sarun@sarun-flowx13:~/os_course/fork$ gcc -o moreforks moreforks.c
sarun@sarun-flowx13:~/os_course/fork$ ./moreforks
parent i = 0
child i = 0
parent i = 1
child i = 1
parent i = 2
child i = 2
sarun@sarun-flowx13:~/os_course/fork$
```

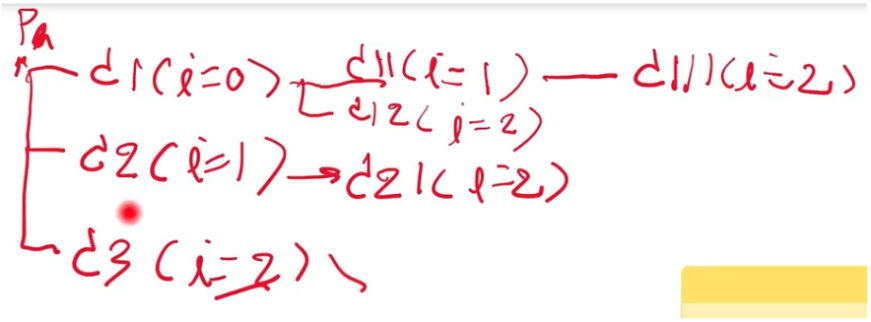
- **Output: 6 lines**
  - 3 from parent
  - 3 from child
- **Processes (include parent): 4 processes**
  - 1 parent
  - 3 child



- 🌱 **Example 2**

```c
int main()
{
  pid_t pid;
  int i;
  for (int i = 0; i < 3; i++) {
    pid = fork();
    if (pid == 0) {
      printf("child i = %d\n", i);
      //exit(0);
    }
    else {
      printf("parent i = %d\n", i);
    }
  }
  return 0;
}
```

- **Processes (include parent):**
  - 8 processes
  - Each process need to fork till i = 2.



> **If you forget to exit, this can cause the problem.**

when you do multiple fork, you need to be careful about this.

- 🌱 **Example 3**

```c
#include <unistd.h>
#include <sys/types.h>

int main()
{
  pid_t pid;
  int i;
  for (int i = 0; i < 3; i++) {
    pid = fork();
    if (pid == 0) {
      printf("child i = %d\n", i);
      exit(0);
    }
    /*else {
      printf("parent i = %d\n", i);
    }*/
  }
  printf("I am parent goodbye\n");
}
```

```
sarun@sarun-flowx13:~/os_course/fork$ gcc -o moreforks moreforks.c
sarun@sarun-flowx13:~/os_course/fork$ ./moreforks
child i = 0
I am parent goodbye
child i = 1
child i = 2
sarun@sarun-flowx13:~/os_course/fork$ ./moreforks
child i = 0
child i = 1
I am parent goodbye
child i = 2
sarun@sarun-flowx13:~/os_course/fork$ ./moreforks
child i = 0
child i = 1
I am parent goodbye
child i = 2
sarun@sarun-flowx13:~/os_course/fork$
```

- **Output: 4 lines**

- 🌱 **Example 4 (introduced to wait)**

```c
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
int main()
{
  pid_t pid;
  int i;
  for (int i = 0; i < 3; i++) {
    pid = fork();
    if (pid == 0) {
      printf("child i = %d\n", i);
      exit(0);
    }
    /*else {
      printf("parent i = %d\n", i);
    }*/
  }
  wait(NULL);
  printf("I am parent goodbye\n");
}
```

- wait is a easy to do some synchronization between parent and child.
- Parent will wait until 1 of its children terminate.

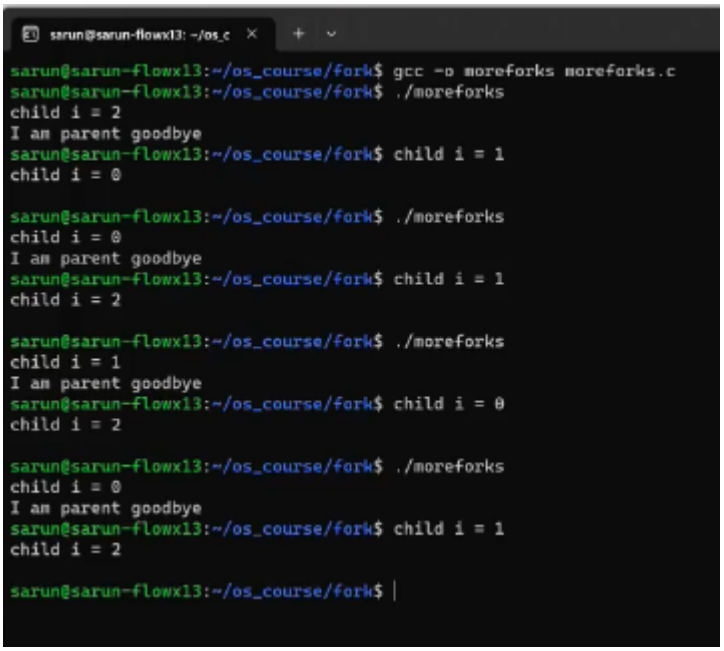
```
sarun@sarun-flowx13: ~/os_c
sarun@sarun-flowx13:~/os_course/fork$ gcc -o moreforks moreforks.c
sarun@sarun-flowx13:~/os_course/fork$ ./moreforks
child i = 0
child i = 1
I am parent goodbye
child i = 2
sarun@sarun-flowx13:~/os_course/fork$ ./moreforks
child i = 0
child i = 1
child i = 2
I am parent goodbye
sarun@sarun-flowx13:~/os_course/fork$ ./moreforks
child i = 0
child i = 2
child i = 1
I am parent goodbye
sarun@sarun-flowx13:~/os_course/fork$ ./moreforks
child i = 0
child i = 1
child i = 2
I am parent goodbye
sarun@sarun-flowx13:~/os_course/fork$
```

Parent can be in any order, except first statement.

- 🌱 **Example 5 (sleeping)**

```c
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
int main()
{
  pid_t pid;
  int i;
  for (int i = 0; i < 3; i++) {
    pid = fork();
    if (pid == 0) {
      srand(getpid());
      sleep(rand() % 5);
      printf("child i = %d\n", i);
      exit(0);
    }
    /*else {
      printf("parent i = %d\n", i);
    }*/
  }
  wait(NULL);
  printf("I am parent goodbye\n");
}
```

- put each child process to sleep for 0-4 seconds.
- getpid() - system call to get process id.
  - we use process id of each child as a seed for the random number
- sleep(rand() % 5) = remainder of number divided by 5.



The point is for this program, we can said that they will have 4 lines of output, and I am parent goodbye never be in first statement.

- 🌱 **Example 6 (parent printed last)**

```c
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
int main()
{
  pid_t pid;
  int i;
  for (int i = 0; i < 3; i++) {
    pid = fork();
    if (pid == 0) {
      srand(getpid());
      sleep(rand() % 5);
      printf("child i = %d\n", i);
      exit(0);
    }
    /*else {
      printf("parent i = %d\n", i);
    }*/
  }
  while(wait(NULL) != -1);
  printf("I am parent goodbye\n");
}

```

```
sarun@sarun-flowx13:~/os_course/fork$ gcc -o moreforks moreforks.c
sarun@sarun-flowx13:~/os_course/fork$ ./moreforks
child i = 0
child i = 2
child i = 1
I am parent goodbye
sarun@sarun-flowx13:~/os_course/fork$ ./moreforks
child i = 2
child i = 0
child i = 1
I am parent goodbye
sarun@sarun-flowx13:~/os_course/fork$
```